FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

1

RANDOM WALK SIMULATION USING AGENTPY

Student Name	Pragamac, Renzel John S.
Section:	AN43
Professor:	Crisola G. Tan
Date:	October 7, 2026

I. BACKGROUND INFORMATION

A **random walk** is a mathematical process that describes a path consisting of a sequence of **random steps**. Random walks are used in many fields, such as physics, biology, economics, and computer science, to model systems that evolve randomly over time.

In this laboratory, we simulate multiple agents moving randomly on a grid using **AgentPy**, a Python library for **agent-based modeling**. Each agent represents an entity that can move one step at a time in a random direction. This simulation helps students understand:

- How **agents move and interact** in a grid-based environment.
- The concept of **bounded movement**, ensuring agents stay within the grid limits.
- What changing **parameters** like the number of agents, grid size, or number of steps affects the system.
- How **agent-based models** can be used to simulate real-world phenomena, such as **diffusion, crowd dynamics, or swarm behavior**.

II. TECHNICAL ASSESSMENT

RANDOM WALK SIMULATION USING AGENTPY

Objective

At the end of this laboratory activity, students are expected to:

1. To understand the concept of **random walks** in a grid environment.
2. To implement a **multi-agent simulation** using Python and AgentPy.
3. To visualize the **movement of agents** on a grid over time.

Materials Needed

- Computer with Python 3.x installed
- Libraries: agentpy, matplotlib
- Code editor (e.g., VSCode, PyCharm, or Jupyter Notebook)

General Instructions

1. This **Technical Assessment (TA4)** must be completed individually.
2. Follow the Task below.
3. Include clear comments in your code to describe its logic.
4. Each program should display the correct input, process, and output.
5. Ensure that your script displays clear and readable outputs in the Command Window.
6. Compile all Python codes and corresponding outputs into one PDF file.
7. Save it as **TA1_lastname.pdf**
   (Example: TA1_Patilano.pdf)

8. Submit the **python file** and **PDF** file.

Activity Instructions

Step 1: Run the Code

1. Open the provided Python code.

Random Walk Simulation (Console-Based)

```
import agentpy as ap
import random
import matplotlib.pyplot as plt
import matplotlib.animation as animation

# -------- Get user input from console --------
num_agents = int(input("Enter number of agents: "))
grid_size = int(input("Enter grid size (e.g., 10 for 10x10): "))
num_steps = int(input("Enter number of steps: "))

# -------- Agent Definition -----------
class RandomWalker(ap.Agent):
    def step(self):
        direction = random.choice([(1,0), (-1,0), (0,1), (0,-1)])
        x, y = self.position
        # Keep agents inside grid
        x = max(0, min(self.model.p.grid_size[0]-1, x + direction[0]))
        y = max(0, min(self.model.p.grid_size[1]-1, y + direction[1]))
        self.position = (x, y)

# -------- Model Definition -----------
class RandomWalkModel(ap.Model):
    def setup(self):
        # Create agents
        self.agents = ap.AgentList(self, self.p.agents, RandomWalker)

        # Initialize random positions
        for agent in self.agents:
            agent.position = (random.randint(0, self.p.grid_size[0]-1),
                              random.randint(0, self.p.grid_size[1]-1))

        # Create grid (optional)
        self.grid = ap.Grid(self, self.p.grid_size, torus=False)
        self.grid.add_agents(self.agents)

    def step(self):
        for agent in self.agents:
            agent.step()

# -------- Parameters from user input -----------
parameters = {
    'agents': num_agents,
    'grid_size': (grid_size, grid_size),
    'steps': num_steps
}
```

```
# -------- Run Model -----------
model = RandomWalkModel(parameters)
model.setup()

# -------- Interactive Animation -----------
fig, ax = plt.subplots()
ax.set_xlim(0, model.p.grid_size[0])
ax.set_ylim(0, model.p.grid_size[1])
ax.set_xticks(range(model.p.grid_size[0]+1))
ax.set_yticks(range(model.p.grid_size[1]+1))
ax.grid(True)

scat = ax.scatter([], [], s=200)  # s = size of agents

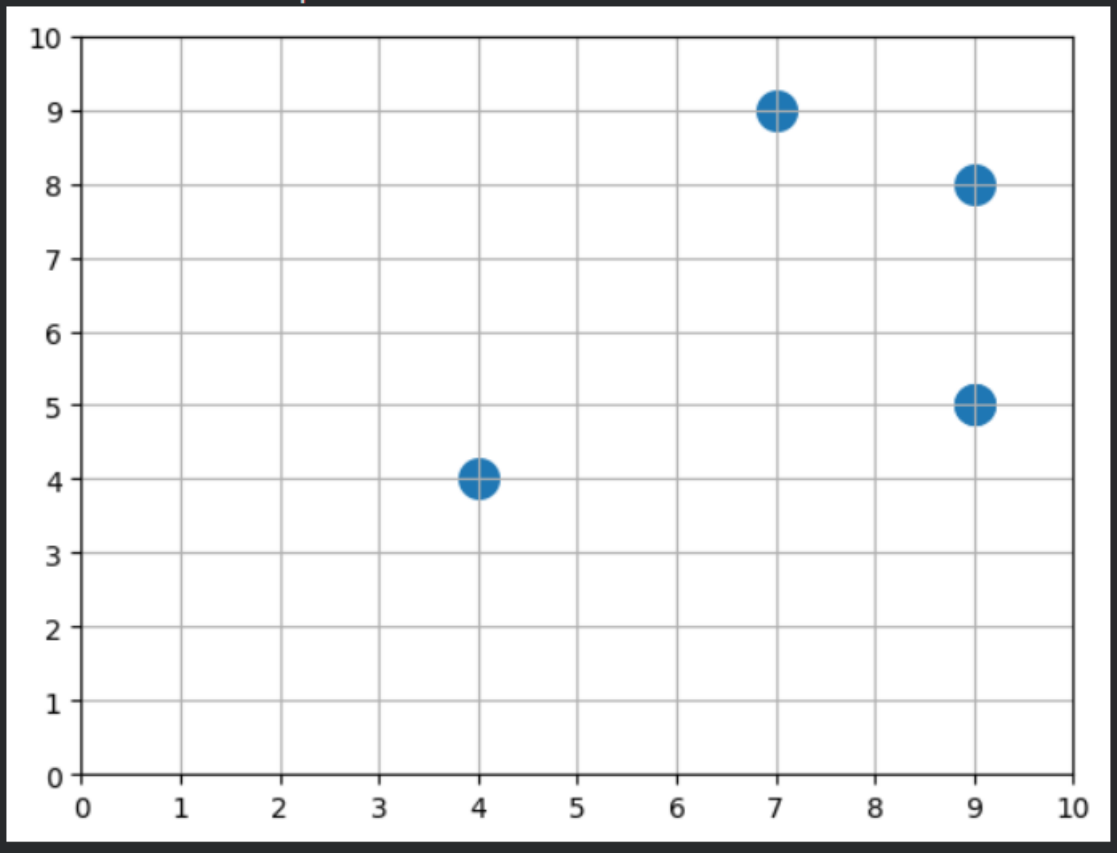
def update(frame):
    model.step()
    x = [agent.position[0] for agent in model.agents]
    y = [agent.position[1] for agent in model.agents]
    scat.set_offsets(list(zip(x, y)))
    return scat,

ani = animation.FuncAnimation(fig, update, frames=model.p.steps, blit=True, repeat=False)
plt.show()
```

2. Run the program in your terminal.
3. Enter the following values when prompted:
    - **Number of agents** (e.g., 5)
    - **Grid size** (e.g., 10 for a 10x10 grid)
    - **Number of steps** (e.g., 20)

These values control the number of agents, the size of the environment, and the simulation duration.

```
Enter number of agents: 5
Enter grid size (e.g., 10 for 10x10): 10
Enter number of steps: 20
```



Step 2: Understand the Agent Behavior

1. Each agent is represented by a dot in the grid.
2. Agents move **randomly** in one of four directions: up, down, left, or right.
3. Agents **cannot leave the grid**; movement is restricted within the grid boundaries.

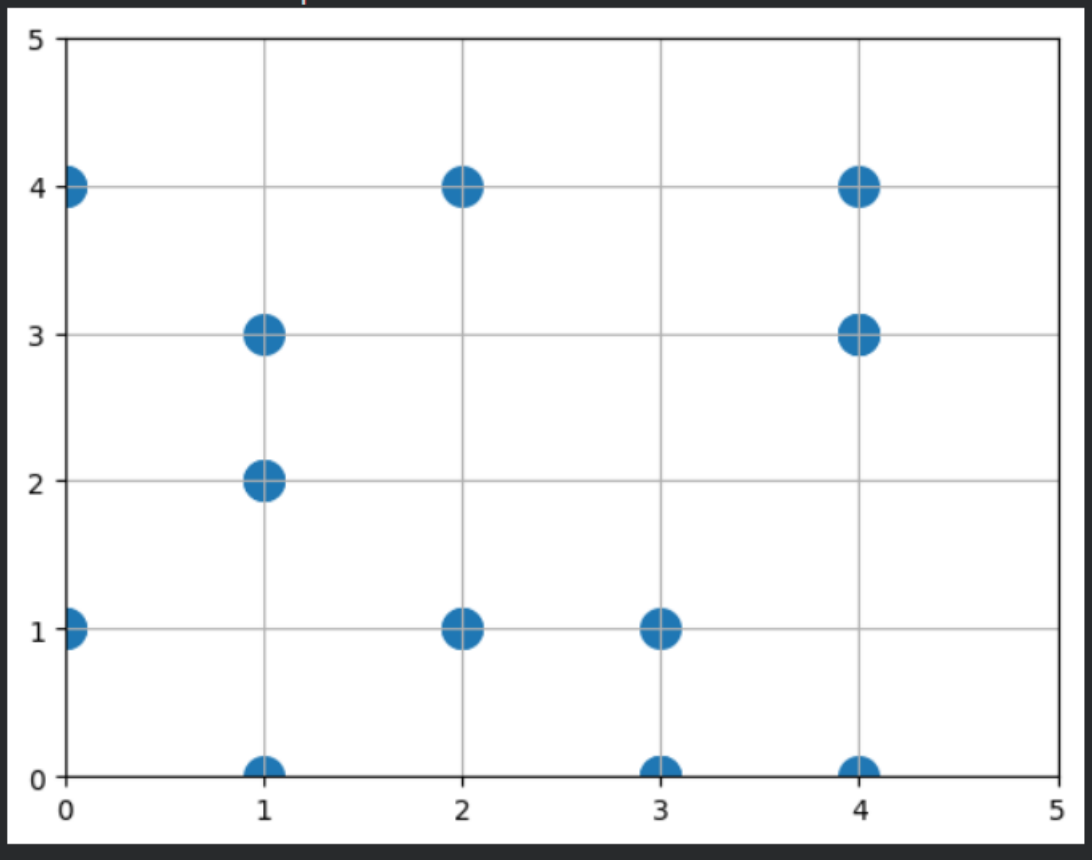
Step 3: Observe the Simulation

1. After running, a **grid with moving agents** is displayed.
2. Each time step updates the position of all agents.
3. Observe:
    - How agents spread across the grid over time
    - Whether agents cluster in certain areas or move freely

Step 4: Experiment with Parameters

- Repeat the simulation with **different values**:
  - **More agents** (e.g., 10 or 20)
  - **Smaller/larger grid sizes**
  - **Longer simulations** (e.g., 50 steps)
- Observe and record how the **agent movement patterns change**.

```
Enter number of agents: 20
Enter grid size (e.g., 10 for 10x10): 5
Enter number of steps: 50
```



Laboratory Tasks

1. **Track agent paths**: Modify the code to **record and display the path of each agent** during the simulation.
2. **Change movement rules**: Make agents **avoid each other** or move in a **preferred direction** and observe the resulting patterns.
3. **Analyze final positions**: Record the **final positions** of all agents and analyze their **distribution**.
4. **Compare scenarios**: Run **two simulations with different parameters** and compare the movement patterns.

Modified (Updated Code - Laboratory Tasks)

CODE

```
import agentpy as ap
import random
import matplotlib.pyplot as plt
import numpy as np

# -------- Get user input from console --------
num_agents = int(input("Enter number of agents: "))
grid_size = int(input("Enter grid size (e.g., 10 for 10x10): "))
num_steps = int(input("Enter number of steps: "))

# -------- Agent Definition ----------
class CustomWalker(ap.Agent):
    def setup(self):
        self.position = (0, 0)
        self.path = []

    def step(self):
        # Record current position before moving
        self.path.append(self.position)

        # Movement Rules
        if self.model.p.bias:
            # Preferred direction: biased towards +x (right) and +y (up)
            # Weights: Right (40%), Up (40%), Left (10%), Down (10%)
            direction = random.choices(
                [(1,0), (0,1), (-1,0), (0,-1)],
                weights=[0.4, 0.4, 0.1, 0.1],
                k=1
            )[0]
        else:
            # Unbiased standard random walk
            direction = random.choice([(1,0), (-1,0), (0,1), (0,-1)])
```

```
        x, y = self.position
        # Keep agents inside grid
        x = max(0, min(self.model.p.grid_size[0]-1, x + direction[0]))
        y = max(0, min(self.model.p.grid_size[1]-1, y + direction[1]))
        self.position = (x, y)

    def record_final_path(self):
        # Record the very last position
        self.path.append(self.position)

# -------- Model Definition -----------
class RandomWalkModel(ap.Model):
    def setup(self):
        # Create agents
        self.agents = ap.AgentList(self, self.p.agents, CustomWalker)

        # Initialize random positions
        for agent in self.agents:
            agent.position = (random.randint(0, self.p.grid_size[0]-1),
                              random.randint(0, self.p.grid_size[1]-1))
            agent.path = [agent.position]

        # Create grid
        self.grid = ap.Grid(self, self.p.grid_size, torus=False)
        self.grid.add_agents(self.agents)

    def step(self):
        self.agents.step()

    def end(self):
        for agent in self.agents:
            agent.record_final_path()

# -------- Run Scenario A: Standard Random Walk -----------
params_a = {
    'agents': num_agents,
    'grid_size': (grid_size, grid_size),
    'steps': num_steps,
    'bias': False
}
model_a = RandomWalkModel(params_a)
model_a.run()
```

```
# -------- Run Scenario B: Biased Walk (Preferred Direction) ----------
params_b = {
    'agents': num_agents,
    'grid_size': (grid_size, grid_size),
    'steps': num_steps,
    'bias': True
}
model_b = RandomWalkModel(params_b)
model_b.run()

# -------- Extract Results for Analysis & Visualization --------
def analyze_model_results(model):
    final_positions = np.array([agent.position for agent in model.agents])
    x_final = final_positions[:, 0]
    y_final = final_positions[:, 1]

    mean_pos = np.mean(final_positions, axis=0)
    var_pos = np.var(final_positions, axis=0)
    return final_positions, x_final, y_final, mean_pos, var_pos

pos_a, x_a, y_a, mean_a, var_a = analyze_model_results(model_a)
pos_b, x_b, y_b, mean_b, var_b = analyze_model_results(model_b)

print("=== SCENARIO A: Standard Random Walk ===")
print(f"Mean final position: {mean_a}")
print(f"Variance of final position: {var_a}\n")

print("=== SCENARIO B: Biased Random Walk (Preferred Up/Right) ===")
print(f"Mean final position: {mean_b}")
print(f"Variance of final position: {var_b}\n")

# -------- Plot and Compare Both Scenarios ----------
fig, axes = plt.subplots(2, 2, figsize=(14, 12))

# Helper to plot paths
def plot_paths(ax, model, title):
    ax.set_title(title)
    ax.set_xlim(-0.5, grid_size - 0.5)
    ax.set_ylim(-0.5, grid_size - 0.5)
    ax.set_xticks(range(grid_size))
    ax.set_yticks(range(grid_size))
    ax.grid(True, linestyle='--', alpha=0.5)

    # Plot paths for each agent
```

```
    for agent in model.agents:
        xs = [p[0] for p in agent.path]
        ys = [p[1] for p in agent.path]
        ax.plot(xs, ys, alpha=0.3, linewidth=1.5)

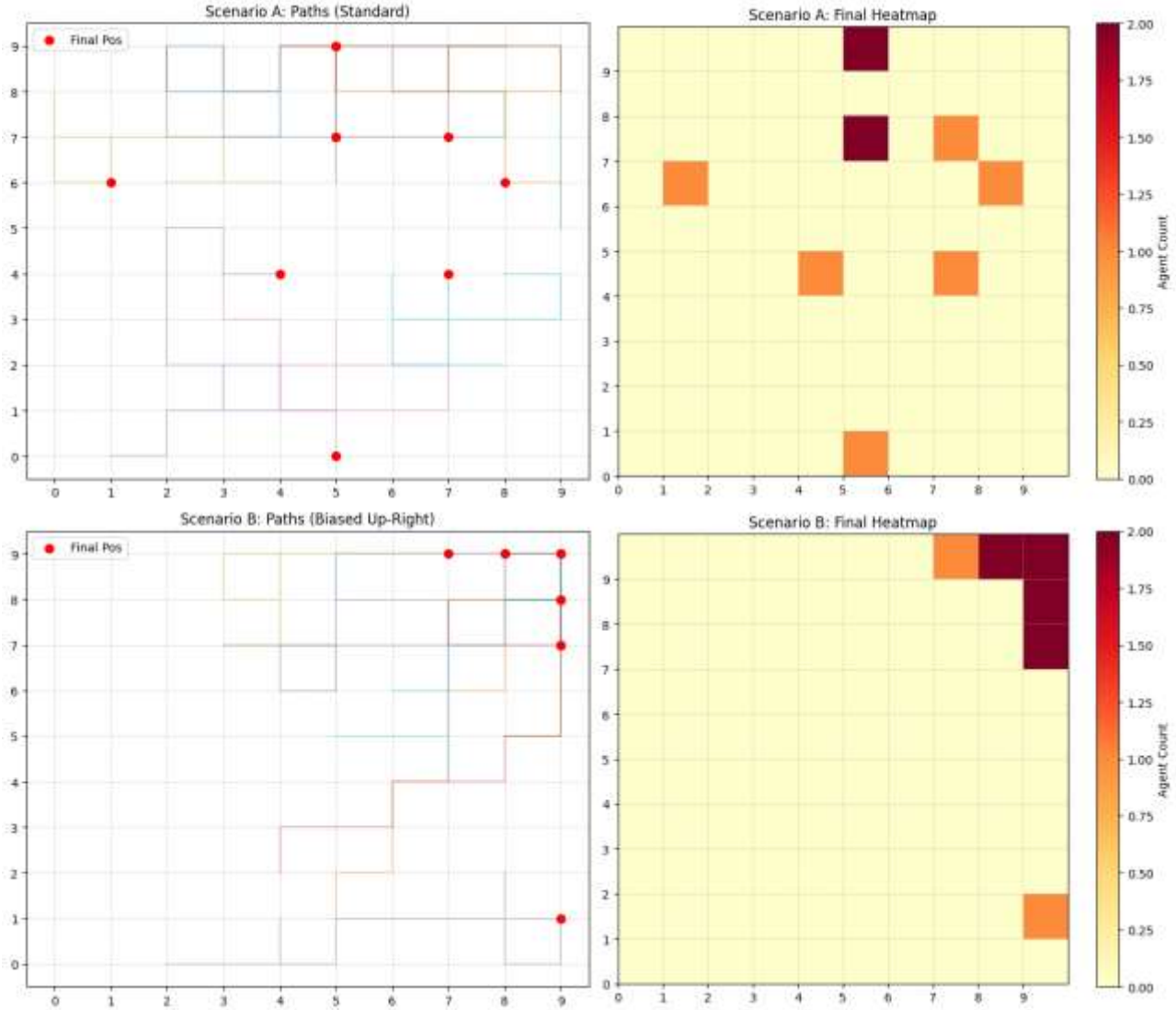
    # Plot final positions
    xs_final = [agent.position[0] for agent in model.agents]
    ys_final = [agent.position[1] for agent in model.agents]
    ax.scatter(xs_final, ys_final, color='red', zorder=5, s=50, label='Final Pos')
    ax.legend()

# Helper to plot final position heatmap/distribution
def plot_heatmap(ax, x_pos, y_pos, title):
    ax.set_title(title)
    counts, xedges, yedges = np.histogram2d(x_pos, y_pos, bins=np.arange(grid_size+1))
    im = ax.imshow(counts.T, origin='lower', cmap='YlOrRd', extent=[0, grid_size, 0, grid_size])
    ax.set_xticks(range(grid_size))
    ax.set_yticks(range(grid_size))
    ax.grid(True, linestyle='--', alpha=0.5)
    fig.colorbar(im, ax=ax, label='Agent Count')

# Plot Scenario A
plot_paths(axes[0, 0], model_a, "Scenario A: Paths (Standard)")
plot_heatmap(axes[0, 1], x_a, y_a, "Scenario A: Final Heatmap")

# Plot Scenario B
plot_paths(axes[1, 0], model_b, "Scenario B: Paths (Biased Up-Right)")
plot_heatmap(axes[1, 1], x_b, y_b, "Scenario B: Final Heatmap")

plt.tight_layout()
plt.show()
```


Scenario A: Paths (Standard)
Final Pos
Scenario A: Final Heatmap
Agent Count
Scenario B: Paths (Biased Up-Right)
Final Pos
Scenario B: Final Heatmap
Agent Count

Step 5: Answer the Questions

1. What happens if the number of agents is **very high** compared to the grid size?
2. How does increasing the number of steps affect the **spread of agents**?
3. Do the agents ever move outside the grid? Why or why not?
4. How can this simulation be used to **model real-life phenomena** (e.g., diffusion, crowd movement)?

Answer the Questions ⇩

1. The grid becomes crowded, while agents will occupy the same grid cells frequently, leading to massive overlapping of agent paths and concentrated clusters on the heatmaps.
2. Increasing the number of steps allows unbiased agents to spread out much further across the grid, while biased agents progressively accumulate and pile up against the grid's outer boundary walls.
3. No, agents never move outside the grid because the code explicitly uses minimum and maximum constraints to keep their coordinates strictly within the grid's dimensions.
4. This simulation can model molecular diffusion (standard random walk), crowd evacuation patterns, or epidemic transmission dynamics (with agent contact rules).